El templo perdido

1. Introducción

Hace tres semanas, la **Dra. Vega** entró sola en un templo recién descubierto en plena selva y no ha vuelto a salir. Su compañero de expedición, un arqueólogo, va a entrar a buscarla. El templo es una red de salas en penumbra, y los rumores de la aldea no animan mucho:

- Hay **fosos de pinchos** ocultos en el suelo, y caer en uno es una muerte segura.
- En alguna sala duerme una **serpiente gigante**, que mataría a quien entrara en su sala.
- Nadie recuerda dónde está la **salida**, aunque desde dentro se ve la luz del día que entra por ella.

El arqueólogo lleva consigo un plano antiguo del templo, que solo indica las salas con escombros, y una **antorcha** que puede lanzar una vez para ahuyentar a la serpiente. Lo demás tendrá que deducirlo de lo que vaya percibiendo.

Y un aviso de la aldea: si la serpiente se despierta, no dejará de perseguir al arqueólogo.

El objetivo del proyecto es construir **agentes** cada vez más capaces de completar esta misión aplicando lo visto en clase:

- Las arquitecturas de agente (Tema 1).
- La búsqueda informada y la búsqueda adversarial (Tema 2).
- La representación del conocimiento y el razonamiento lógico, con programación declarativa en ASP (Tema 3).
- La coordinación de **varios agentes** que cooperan y se reparten las tareas según su rol (Parte 4).

Se proporciona un **motor de juego** ya hecho, con el templo, las percepciones, una interfaz gráfica y los tests. El trabajo consiste en programar la inteligencia del agente.

2. El templo

El templo es una rejilla de 6×6 salas. Cada celda se identifica como (f, c), con fila y columna empezando en 1; la fila 1 es la de arriba. El arqueólogo entra siempre por la celda $(1, 1)$.

En el templo hay:

- **3 fosos** de pinchos (F).
- **1 serpiente** dormida (S).

- La **Dra. Vega** (V). En las expediciones de la Parte 2a hay además dos ayudantes perdidos: hay que rescatarlos a todos.
- La **salida** (E).
- Varias celdas con **escombros** ($*$).

Fosos, serpiente, Dra. Vega y salida ocupan celdas distintas. Las celdas con escombros se pueden atravesar, pero entrar en ellas cuesta 3 en lugar de 1.

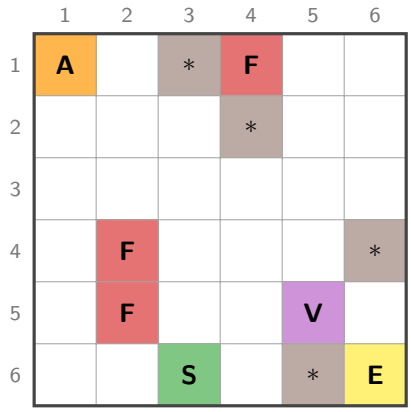


Figura 1. Templo de ejemplo (`mapas/ejemplo_enunciado.txt`). A: entrada $(1, 1)$; F: foso; S: serpiente; V: Dra. Vega; E: salida; $*$: escombros.

El agente **no conoce** dónde están los fosos, la serpiente, la Dra. Vega ni la salida. Sí conoce:

- El tamaño del templo.
- Las celdas con escombros, porque vienen en el plano.
- El número de fosos, que también indica el plano.
- Su propia posición.
- Lo que percibe en cada momento.

Cada partida usa un templo **aleatorio** (`generar_mundo(semilla)`), y la misma semilla genera siempre el mismo templo. El generador garantiza dos cosas:

- La celda inicial y sus vecinas no tienen peligros.
- El templo se puede resolver *solo con lógica*: siempre existe un camino por celdas que se pueden *demostrar* seguras hasta la Dra. Vega y hasta la salida, sin necesidad de la antorcha.

3. Acciones, costes y puntuación

Acción	Efecto	Coste
`ARRIBA`, `ABAJO`, `IZQUIERDA`, `DERECHA`	Moverse a la celda adyacente. Si hay pared, no se mueve. Entrar en una celda con foso, o con la serpiente si sigue ahí, mata al arqueólogo. Al entrar en la celda de la Dra. Vega, se la rescata automáticamente y desde ese momento viaja con el arqueólogo.	1 ó 3
`LANZAR_ARRIBA`, ..., `LANZAR_DERECHA`	Lanzar la antorcha (**un solo uso**) a la celda adyacente en esa dirección. Si la serpiente está allí, huye: se oye un **chillido** y la celda pasa a ser transitable. Si allí está la Dra. Vega, no le pasa nada.	1
`SALIR`	Si el arqueólogo está en la salida, termina la partida: con éxito si lleva a la Dra. Vega y como fracaso si no. En cualquier otra celda no ocurre nada.	1

- **Coste de moverse:** es el de la celda de *destino*: 1, o 3 si tiene escombros.
- **Puntuación de una partida:** $+1000$ si se sale con la Dra. Vega, -1000 si se muere, menos el coste acumulado.
- **Abandonar:** un agente puede rendirse devolviendo `None`. No pierde los 1000 puntos de la muerte, pero tampoco gana nada.

4. Percepciones

En cada paso el agente recibe un `Percepto` con ocho valores booleanos. Una celda *adyacente* es la que está a distancia Manhattan 1: arriba, abajo, izquierda o derecha.

$$\mathit{percepto}(s) = [\,\mathit{Corriente},\ \mathit{Siseo},\ \mathit{Luz},\ \mathit{Pared}_{\uparrow},\ \mathit{Pared}_{\downarrow},\ \mathit{Pared}_{\leftarrow},\ \mathit{Pared}_{\rightarrow},\ \mathit{Chillido}\,]$$

Percepción	Se percibe cuando...
`corriente`	hay un foso en alguna celda adyacente (corriente de aire).
`siseo`	la serpiente está en una celda adyacente (y no ha huido).
`luz`	la salida está en la celda actual *o* en una adyacente (luz del día).
`pared_arriba`, ...	hay una pared del templo en esa dirección.
`chillido`	la antorcha que se acaba de lanzar ha ahuyentado a la serpiente.

Por ejemplo, en la celda inicial del templo de la Figura 1 el percepto es $[0, 0, 0, 1, 0, 1, 0, 0]$: no hay ni corriente, ni siseo, ni luz, y hay paredes arriba y a la izquierda.

Cada paso, el agente recibe una `Observacion` con estos campos:

- `posicion`: la celda en la que está.
- `percepto`: lo que percibe en ella.
- `rescatada`: si la Dra. Vega (y, en su caso, todos los perdidos) va ya con él.
- `antorcha`: si todavía la conserva.

- `tamano`: el tamaño del templo.
- `escombros`: las celdas con escombros.

No hay ninguna percepción que indique «esta celda es la salida»: o se deduce, o se prueba `SALIR`.

5. El código proporcionado

```
templo-perdido/
  templo.py             programa principal (jugar / ver a los agentes)
  evaluar.py            estadísticas sobre muchas partidas
  agente/               CÓDIGO A IMPLEMENTAR: solo se modifican estos ficheros
    problema.py         Parte 2a  ProblemaRescate, ProblemaRuta
    heuristicas.py      Parte 2a  heuristica_rescate, heuristica_ruta
    adversarial.py      Parte 2b  minimax, alfabeta, expectimax, evaluación
    templo.lp           Parte 3   el conocimiento del templo en ASP
    agente_logico.py    Parte 3   AgenteLogico.decidir
    expedicion.lp       Parte 4   los repartos válidos de la expedición en ASP
    coordinador.py      Parte 4   elegir el mejor reparto con A*
  engine/               motor del juego (NO modificar); incluye busqueda.py
                        (bfs, dfs, ucs, astar), huida.py (la persecución),
                        referencia.py (AgenteExplorador) y expedicion.py
                        (la expedición de la Parte 4)
  mapas/                templos de ejemplo en formato texto
  tests/                autograder con pytest (NO modificar)
```

Instalación y uso

Primero, uno de los dos miembros de la pareja crea un repositorio **privado** a partir de la plantilla de GitHub de la asignatura (botón *Use this template* → *Create a new repository*, con visibilidad *Private*) y añade como colaboradores al otro miembro de la pareja y al profesor. **No se debe hacer un *fork***: un fork de un repositorio público es público y no se puede hacer privado. Después, cada uno clona el repositorio de la pareja:

```
git clone https://github.com/<usuario>/<repositorio>.git
cd <repositorio>
uv venv                             # crea .venv (uv: https://docs.astral.sh/uv/)
source .venv/bin/activate           # Windows: .venv\Scripts\activate
uv pip install -r requirements.txt

python templo.py                                    # jugar con el teclado
python templo.py --semilla 42                       # templo concreto (reproducible)
python templo.py --mapa ejemplo_enunciado           # templo de la carpeta mapas/
python templo.py --modo explorador                  # agente de referencia
python templo.py --modo busqueda --algoritmo astar  # Parte 2a: plan óptimo
python templo.py --modo busqueda --perdidos 3       # Parte 2a: expedición
python templo.py --modo busqueda --toroidal         # Parte 2a: templo toroidal
python templo.py --modo huida --algoritmo alfabeta  # Parte 2b
python templo.py --modo huida --algoritmo expectimax --serpiente estocastica
python templo.py --modo huida --algoritmo manual    # Parte 2b: huir con el teclado
python templo.py --modo logico                      # Parte 3: agente lógico
python templo.py --modo logico --texto              # cualquier modo, sin GUI
python templo.py --modo expedicion                  # Parte 4: dos arqueólogos
python templo.py --modo expedicion --egoistas       # Parte 4: sin coordinarse
python -m clingo agente/expedicion.lp mapas/expedicion_roles.lp 0
pytest                                              # autograder (o pytest -m parte2b)
```

```
python evaluar.py --agente logico --partidas 200    # estadísticas
```

En el modo manual:

- El arqueólogo se mueve con las flechas o con WASD.
- Para lanzar la antorcha, se pulsa espacio y luego una flecha.
- `X` ejecuta `SALIR`.
- `V` muestra u oculta las deducciones del `AgenteLogico` sobre cada celda: ✓ segura, F?/F! foso posible o seguro, S?/S! serpiente, E?/E! salida.

Interfaces principales del motor

- **Agentes** (`engine.juego.Agente`): implementan `actuar(obs) -> Accion`. Pueden devolver `None` para abandonar.
- **Problemas de búsqueda** (`engine.busqueda_base.ProblemaBusqueda`): implementan tres métodos: `estado_inicial()`, `es_objetivo(e)` y `sucesores(e)`. `sucesores` devuelve una lista de tuplas `(accion, estado, coste)`. Los algoritmos de `engine/busqueda.py` piden los sucesores con `problema.expandir(e)`, que además cuenta los nodos expandidos.
- **Conocimiento en ASP** (`engine.asp.ConocimientoASP`): convierte lo que observa el agente en hechos, los junta con `agente/templo.lp` y pregunta a clingo. Ofrece `cautas()` (lo que se deduce), `bravas()` (lo posible) y `seguras()`.
- **La expedición** (`engine.expedicion`): `repartos_validos(mundo)` junta los hechos de la expedición con `agente/expedicion.lp` y devuelve *todos* los modelos de clingo; `ejecutar_expedicion` simula a los dos arqueólogos a la vez.

6. Parte 2a — Modelado y heurísticas (25 %)

En esta parte se supone que el arqueólogo ha encontrado un **plano completo** del templo. Todo es conocido, así que la misión se convierte en un problema de búsqueda clásico: encontrar la secuencia de acciones *de menor coste* que rescata a todos los perdidos y sale del templo. El plan debe terminar con `SALIR`.

Los algoritmos (`bfs`, `dfs`, `ucs` y `astar`) ya se programaron en la práctica de pathFinding, así que vienen hechos en `engine/busqueda.py`. Aquí el trabajo es **formular** el problema y **diseñar la heurística**.

- **El estado.** Mientras se juega no solo cambia la posición: también cambian otras cosas que afectan a lo que se puede hacer después (¿a quién se ha rescatado ya? ¿queda antorcha? ¿sigue la serpiente en su sitio?). En el mapa `pasillo_serpiente`, el único camino pasa por la serpiente y el plan óptimo necesita la antorcha.
- **Expediciones.** En algunos templos no hay una sola persona perdida sino **tres** (la Dra. Vega y dos ayudantes: varias V en el mapa, como en `mapas/expedicion.txt` o con `--perdidos 3`). El orden de rescate importa, y una heurística que solo mire la salida o el perdido más cercano se queda corta.
- **Templo toroidal.** En esta variante (`mundo.toroidal`, opción `--toroidal`) no hay paredes: quien sale por un borde aparece por el opuesto. Por ejemplo, ir a la izquierda desde $(3, 1)$ lleva a $(3, 6)$. `mundo.envolver(celda)` devuelve la celda equivalente dentro de la rejilla. En `mapas/toroidal.txt`, el plan óptimo cuesta 17 con paredes y 9 cruzando los bordes.

Se pide:

a) `ProblemaRescate` (`agente/problema.py`): formular el problema para cualquier número de perdidos (`mundo.perdidos`).

   (I) Definir el estado, el estado inicial, el test de objetivo y los sucesores con sus costes.

   (II) No generar acciones que maten al arqueólogo.

b) `heuristica_rescate`: una heurística **admisible** para A*. Los tests exigen que A* expanda como mucho el 60 % de los nodos de UCS con un perdido, y el 30 % con tres.

c) Que `ProblemaRescate` y `heuristica_rescate` funcionen también en templos **toroidales**, y que la heurística siga siendo admisible.

d) `ProblemaRuta` y `heuristica_ruta`: ir desde una celda hasta la más cercana de un conjunto de celdas objetivo, pisando solo celdas permitidas. Lo usará el agente de la Parte 3.

7. Parte 2b — La serpiente despierta (25 %)

Al rescatar a la Dra. Vega, el ruido despierta a la serpiente, que empieza a **perseguir** al arqueólogo hasta la salida. Ahora el entorno tiene *otro agente* con intereses opuestos: es un problema de **búsqueda adversarial**. El mapa sigue siendo conocido, como en la Parte 2a.

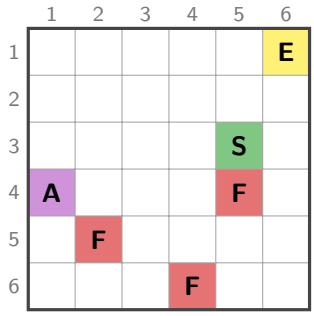


Figura 2. `mapas/huida.txt`: el arqueólogo (A) sale de la celda de la Dra. Vega. Si corre directo a la salida (E), la serpiente (S) le corta el paso; mirando unas jugadas hacia delante, sí escapa.

Reglas de la persecución

- **Turnos:** se juega por turnos, primero el arqueólogo y luego la serpiente.
- **Posiciones iniciales:** el arqueólogo empieza en la celda de la Dra. Vega, y la serpiente en la suya.
- **Fin de la partida:**
  - El arqueólogo **gana** al llegar a la salida.
  - **Pierde** si coincide en una celda con la serpiente.
  - A los 60 turnos, empate.
- **Movimientos:** nadie pisa fosos. La serpiente es algo más lenta: descansa en sus turnos 3, 6, 9...
- **Antorcha:** lanzarla contra la serpiente cuando está adyacente la deja **aturdida** 3 turnos. Un solo uso.

- **Utilidad para el arqueólogo:** $+1000 -$ turnos si escapa, -1000 si lo atrapan y 0 si hay empate.
- **Dos tipos de serpiente:**
  - La **voraz** siempre se acerca todo lo posible.
  - La **estocástica** lo hace con probabilidad 0.7 y, si no, se mueve al azar.

Antes de programar, conviene jugar la persecución a mano con `python templo.py --modo huida --algoritmo manual` (flechas para moverse, espacio y una flecha para lanzar la antorcha): la serpiente mueve sola después de cada jugada.

El motor (`engine/huida.py`) proporciona un `EstadoHuida` con estos métodos:

- `jugador()`: quién mueve.
- `acciones()` y `sucesor(a)`: las acciones legales y el estado al que lleva cada una.
- `es_terminal()` y `utilidad()`.
- `distribucion_serpiente()`: el modelo de azar de la serpiente estocástica.
- `distancia(mundo, a, b)`: la distancia real sin pisar fosos.

Se pide (`agente/adversarial.py`):

a) `minimax(estado, profundidad, evaluar)`: devuelve la mejor acción del arqueólogo suponiendo una serpiente *óptima*. La profundidad cuenta jugadas del arqueólogo; al agotarla se usa la función de evaluación.

b) `alfabeta`: la misma decisión que minimax, con poda alfa-beta. Los tests exigen que genere como mucho el 60 % de los nodos de minimax. En los dos, las acciones se recorren en el orden de `estado.acciones()` y, si hay empate, se elige la *primera* de valor máximo.

c) `expectimax`: la serpiente se modela como *azar*, con `distribucion_serpiente()`.

d) `funcion_evaluacion(estado)`: valora los estados no terminales. Con alfa-beta a profundidad 3 debe escapar de la serpiente voraz en al menos el 80 % de los templos en los que huir en línea recta fracasa.

8. Parte 3 — Agente lógico con ASP (35 %)

Se vuelve a la situación real: el templo está a oscuras y el agente solo sabe lo que ha percibido. Ahora tiene que **razonar** para decidir qué celdas son *seguras*, es decir, cuáles tiene la *certeza* de que no contienen ni un foso ni la serpiente.

Como referencia, el motor incluye un `AgenteExplorador` (`engine/referencia.py`). Es un agente **basado en modelo**, sin lógica formal:

- Recuerda las celdas visitadas.
- Considera segura una celda solo si es vecina de una celda visitada sin corriente ni siseo.
- Explora en profundidad, retrocediendo cuando no le quedan opciones.

No muere nunca, pero se rinde en uno de cada cuatro templos. Es la **referencia que el agente de esta parte debe superar**.

El conocimiento del templo se escribe de forma **declarativa**, en **Answer Set Programming** (ASP), y lo resuelve **clingo**, el mismo que se ha visto en clase. No hay que programar ningún algoritmo de inferencia: basta con describir *qué* es verdad en el templo, y clingo encuentra todos los mundos posibles compatibles con lo observado.

Cómo se usa ASP aquí

- Cada **modelo** (answer set) del programa es un templo posible: una forma de colocar fosos, serpiente y salida que encaja con todo lo percibido.
- El motor (`engine/asp.py`) pide a clingo dos cosas:
  - Las **consecuencias cautas**, ciertas en *todos* los modelos. Eso es exactamente la consecuencia lógica: lo que se *deduce*. En la GUI se ve como ✓, F!, S! o E!.
  - Las **consecuencias bravas**, ciertas en *algún* modelo. Eso es lo *posible*: F?, S? o E?.
- Una celda es segura si `segura(F,C)` es una consecuencia cauta.

Hechos que añade el motor

Los hechos no hay que escribirlos: el motor los añade según lo que observa el agente.

Hecho	Significado
`tamano(N)`, `num_fosos(K)`	El templo es $N \times N$ y el plano dice que hay K fosos.
`visitada(F,C)`	El arqueólogo ha estado (vivo) en (F, C).
`percibe(P,F,C)`, `no_percibe(P,F,C)`	En (F, C) se percibe, o no, $P \in \{$`corriente`, `siseo`, `luz`$\}$.
`antorcha(F,C,si|no)`	Se lanzó la antorcha a (F, C) y se oyó (o no) un chillido.
`huida`	La serpiente ha huido.
`no_es_salida(F,C)`	Se intentó `SALIR` en (F, C) sin éxito.

El programa debe definir, con su `#show`, los predicados `foso/2`, `serpiente/2`, `salida/2` y `segura/2`.

El conocimiento que hay que expresar

Para una celda x con vecinas $v_1, \ldots, v_k$:

$$Co_x \leftrightarrow F_{v_1} \vee \cdots \vee F_{v_k} \qquad Si_x \leftrightarrow S_{v_1} \vee \cdots \vee S_{v_k} \qquad L_x \leftrightarrow E_x \vee E_{v_1} \vee \cdots \vee E_{v_k}$$

Además:

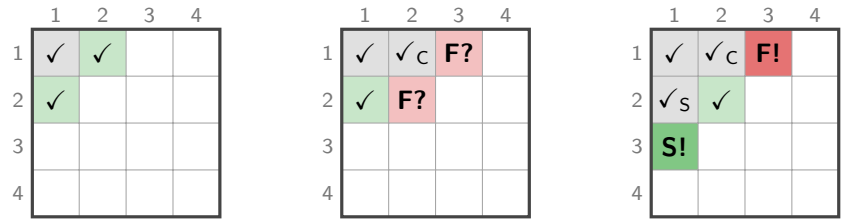
- Hay *exactamente* K fosos, una serpiente y una salida, en celdas distintas.
- Donde el arqueólogo ha estado vivo no hay foso, ni serpiente (salvo que haya huido).
- El chillido indica dónde estaba la serpiente, y un `SALIR` fallido descarta esa celda como salida.

En ASP, cada «↔» se escribe como una regla que *define* cuándo hay corriente, siseo o luz, y una **restricción** que descarta los modelos que no coinciden con lo percibido. Por ejemplo, para la corriente:

```
hay(corriente,F,C) :- ady(F,C,X,Y), foso(X,Y).
:- percibe(P,F,C), not hay(P,F,C).
```

Un ejemplo de razonamiento

Templo de 4×4 (`mapas/razonamiento_4x4.txt`). En la figura, gris es una celda visitada, verde una segura, y los subíndices indican lo percibido (C: corriente, S: siseo).



(a) nada en $(1,1)$ (b) corriente en $(1,2)$ (c) siseo en $(2,1)$

(a) En $(1,1)$ no se percibe nada, así que sus vecinas $(1,2)$ y $(2,1)$ son seguras.

(b) El arqueólogo entra en $(1,2)$ y percibe corriente: hay un foso en $(1,3)$ o en $(2,2)$, pero aún no se sabe en cuál.

(c) Vuelve a $(1,1)$ y entra en $(2,1)$, donde percibe un siseo. La serpiente podría estar en $(1,1)$, $(2,2)$ o $(3,1)$:

- La celda $(1,1)$ queda descartada porque ya la visitó.
- La celda $(2,2)$ queda descartada porque, si la serpiente estuviera ahí, habría oído el siseo desde $(1,2)$.
- Luego la serpiente está en $(3,1)$.

Además, como en $(2,1)$ no hay corriente, $(2,2)$ no tiene foso. Y como en $(1,2)$ sí había corriente, el foso tiene que estar en $(1,3)$.

Estas conclusiones combinan lo percibido en *varias* celdas, no solo en la última. Con un `templo.lp` correcto, clingo llega solo a estas mismas conclusiones: es lo que comprueba el test `test_razonamiento_4x4`.

Se pide:

a) `agente/templo.lp`: el conocimiento del templo en ASP. El esqueleto ya trae el dominio (`celda/2` y `ady/4`) y cuatro huecos `% TODO`:

   (I) Dónde puede estar cada cosa (reglas de elección con cardinalidad).

   (II) Qué produce cada cosa en las celdas vecinas.

   (III) Las restricciones que obligan a coincidir con lo observado.

   (IV) Cuándo una celda es segura.

b) `AgenteLogico.decidir()` (`agente/agente_logico.py`). El resto del agente viene hecho: `observar` y las consultas a clingo. La estrategia:

   - Explorar la celda segura más cercana, desplazándose con `ProblemaRuta` y A* *solo por celdas seguras*.
   - Con la Dra. Vega, ir a la salida: deducida, o probando `SALIR` en las celdas donde es posible.
   - Si ya no queda nada seguro, usar la antorcha contra la serpiente.
   - El agente no debe morir **nunca**.

c) **Modo manual guiado:** con el programa terminado, `python templo.py` muestra en cada celda lo que se deduce (cauto) y lo que es posible (bravo). Conviene usarlo para comprobar las deducciones de `templo.lp`.

9. Parte 4 — La expedición: varios agentes (15 %)

La Dra. Vega ya está a salvo, pero su equipo de **cinco exploradores** sigue dentro del templo. Algunos están heridos y otros han quedado atrapados bajo los escombros. Esta vez entra una expedición de rescate con **dos arqueólogos**, Indy y Lara, y cada uno lleva un equipo distinto: su **rol**. El mapa es conocido, como en la Parte 2a.

Ahora hay *varios agentes* en el entorno y comparten un mismo objetivo: es un sistema **multiagente cooperativo**. La pregunta clave es **quién rescata a quién**.

Arqueólogo	Equipo (rol)	Qué le permite
`indy`	botiquín	Es el único que puede rescatar a un perdido **herido** (H).
`lara`	pico	Es la única que puede liberar a un perdido **atrapado** (X), que además está en una celda con escombros.

- Los perdidos normales (V) los puede rescatar cualquiera.
- Cada arqueólogo puede sacar como mucho a **3** perdidos (`capacidad`).
- Los dos entran por $(1, 1)$ y se mueven a la vez, cada uno a su ritmo. Su reloj es su coste acumulado.
- La expedición termina cuando sale el *último*. Su duración, el coste del arqueólogo que más tarda, es el **makespan**, y es lo que hay que minimizar.

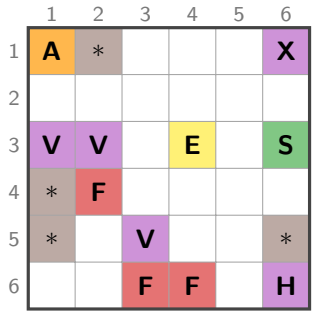


Figura 3. `mapas/expedicion_roles.txt`. H: herido (solo Indy); X: atrapado (solo Lara); V: cualquiera.

Coordinar en tres pasos

1. **clingo genera los repartos posibles** (`agente/expedicion.lp`). Cada modelo es un reparto que cumple todas las reglas: los roles, la capacidad, etc. El motor pide *todos* los modelos.
2. **A* decide cuál es el mejor** (`agente/coordinador.py`). Para cada reparto, cada arqueólogo planifica su parte con el `ProblemaRescate` de la Parte 2a, limitado a *sus* perdidos, y se calcula el makespan. Se elige el reparto de menor makespan.
3. **El motor ejecuta los dos planes a la vez**, y cada arqueólogo solo rescata a los perdidos que le tocan.

Es el mismo patrón ASP que en la Parte 3: *generar* con reglas de elección y *descartar* con restricciones. En la Parte 3, cada modelo es un *templo posible*; aquí, cada modelo es un *plan de equipo posible*. Clingo decide qué es *posible* y la búsqueda decide qué es *mejor*, así que no hace falta ninguna instrucción de optimización de ASP.

Hechos que añade el motor

Hecho	Significado
`arqueologo(indy)`, `arqueologo(lara)`	Los dos arqueólogos.
`equipo(indy,botiquin)`, `equipo(lara,pico)`	El equipo (rol) de cada uno.
`perdido(p1)`, …, `perdido(p5)`	Los perdidos, numerados por filas: `p1` es el de más arriba.
`herido(P)`, `atrapado(P)`	`P` necesita botiquín, o pico.
`capacidad(K)`	Cada arqueólogo saca como mucho a `K` perdidos.

El programa debe definir `rescata(A,P)`, que significa «el arqueólogo `A` rescata al perdido `P`», y mostrarlo con `#show`. Se puede probar directamente en la terminal, con los hechos del mapa de la Figura 3:

```
$ python -m clingo agente/expedicion.lp mapas/expedicion_roles.lp 0
Answer: 1
rescata(lara,p1) rescata(lara,p2) rescata(indy,p3) rescata(indy,p4) rescata(indy,p5)
...
Models       : 6
```

La referencia: arqueólogos egoístas

`python templo.py --modo expedicion --egoistas` muestra a los mismos dos arqueólogos *sin coordinarse* (`PoliticaEgoista`). Cada uno va a por el perdido más cercano que puede rescatar, sin saber qué hace el otro. En el mapa de la Figura 3, Indy se llena con los tres V de la izquierda y el herido de $(6, 6)$, al que solo él podía rescatar, se queda en el templo.

Se pide:

a) `agente/expedicion.lp`: los repartos válidos en ASP. Son cinco huecos `% TODO`, del mismo tipo que los puzzles de clase:

   (I) Cada perdido lo rescata exactamente un arqueólogo (regla de elección).

   (II) Un herido solo puede rescatarlo quien lleva botiquín (restricción).

   (III) Un atrapado solo puede rescatarlo quien lleva pico (restricción).

   (IV) Nadie rescata a más de `K` perdidos (restricción con cardinalidad, como `num_fosos` en `templo.lp`).

   (V) Los dos arqueólogos trabajan: nadie se queda sin rescatar a nadie.

   Los tests comparan los modelos obtenidos con todos los repartos válidos: no puede sobrar ni faltar ninguno.

b) `agente/coordinador.py`, que usa el `ProblemaRescate` y la heurística de la Parte 2a:

   (I) `plan_individual(mundo, perdidos)` y `coste_individual(mundo, perdidos)`: el plan óptimo de un arqueólogo que solo rescata a `perdidos`, y su coste.

   (II) `elegir_reparto(mundo, repartos)`: el reparto de menor makespan.

10. Entrega

Entrega:

- **Modalidad:** por parejas.
- **Repositorio:** el código inicial se publica como repositorio-plantilla de GitHub. Cada pareja trabaja en un repositorio **privado** creado con *Use this template* (**nunca un fork**): uno de los dos lo crea y añade como colaboradores al otro miembro y al profesor.
- **Entregable:** el repositorio privado de la pareja. Se corrige la carpeta `agente/` de la rama `main`.
- **Ejecución:** el código debe funcionar con el motor *original*. No se puede modificar nada fuera de `agente/`: se corregirá sobre una copia limpia de `engine/` y `tests/`.
- **Librerías:** solo la librería estándar de Python, `clingo` (incluido en `requirements.txt`) y `numpy` si se necesita. No se pueden usar librerías de búsqueda ni de lógica.
- **Defensa oral:** el **2 de diciembre de 2026**, obligatoria para los dos miembros de la pareja. En ella se evalúa cada parte (ver la sección de evaluación).

No se permite compartir código entre parejas.

11. Evaluación

Bloque	Peso	Qué se valora
Parte 2a — Modelado y heurísticas	25 %	Tests de la Parte 2a, formulación del estado, admisibilidad y calidad de la heurística, y calidad del código.
Parte 2b — Adversarial	25 %	Tests de la Parte 2b, minimax, alfa-beta y expectimax, función de evaluación y calidad del código.
Parte 3 — Agente lógico	35 %	Tests de la Parte 3, comportamiento del agente (tasa de éxito, muertes y coste), representación en `templo.lp` y calidad del código.
Parte 4 — Multiagente	15 %	Tests de la Parte 4, reglas de `expedicion.lp`, coordinador y calidad del código.

La nota de cada parte se fija en una **defensa oral** obligatoria, el **2 de diciembre de 2026**; los turnos se publicarán en Moodle. En ella, cada miembro de la pareja responde de forma *individual* sobre el código entregado, las decisiones de diseño y los conceptos de la asignatura que aplica.

- **Nota individual:** la defensa se califica por separado para cada miembro de la pareja.
- **No saber explicar el código:** si un miembro no es capaz de explicar el código o las decisiones de la entrega, su nota del proyecto podrá reducirse, independientemente del resultado de los tests.
- **No presentarse:** quien no se presente a la defensa sin causa justificada tendrá el proyecto como no presentado.

Los tests públicos (`pytest`) sirven para comprobar el trabajo, pero aprobarlos no garantiza la nota de esa parte.

¡Suerte, y cuidado con los fosos!